Technical Note for the Safe Implementation of a VQE-Based De-escalation Portfolio Simulator

A Fujitsu QuLacs-Oriented Project Guide for Quantum-Assisted Scenario Optimization

Ahmad R. T. Nugraha, Muhammad Syauqi Fittuqo, Donny Dwiputra,
Taufiq Wirahman, and Meng E. Ong

May 26, 2026

Abstract

This technical note describes a possible implementation of a quantum-assisted conflict-prevention project using the variational quantum eigensolver (VQE). The project is formulated as a controlled simulation study: given a synthetic signed network of geopolitical tensions and a set of civilian de-escalation measures, construct a quadratic optimization problem whose solutions correspond to cost-constrained portfolios of preventive actions. The resulting QUBO is mapped to an Ising Hamiltonian and solved using VQE or QAOA-like Hamiltonian variational circuits implemented with QuLacs-style programming from Fujitsu. The project outputs include risk-proxy reduction curves, intervention-cost Pareto frontiers, solution diversity, robustness under synthetic shocks, and comparisons against classical baselines such as greedy optimization and simulated annealing. This note gives step-by-step mathematical derivations, proposed implementation details, recommended code structure, QuLacs circuit construction patterns, observables, sampling, post-processing, evaluation metrics, and realistic limitations.

Contents

1 Brief Intro. **4**

2 Scope: What This Project Does and Does Not Do **4**
2.1 What the project does . . . 4
2.2 What the project does not do . . . 5

3 Proposed Project Architecture **5**
3.1 Primary modeling choice . . . 5
3.2 Data layers . . . 6

4 Mathematical Model **6**
4.1 Synthetic country network . . . 6
4.2 Block-structured country generator . . . 6
4.3 Scenario set . . . 7
4.4 Candidate de-escalation actions . . . 7
4.5 Action effectiveness matrix . . . 8

5 From Scenario Risk to QUBO **8**
5.1 Residual tension model . . . 8
5.2 Second-order QUBO approximation . . . 9
5.3 Scenario aggregation . . . 9
5.4 Budget penalty . . . 9
5.5 Optional incompatibility penalties . . . 10

6 QUBO to Ising Hamiltonian **10**
6.1 Binary-spin mapping . . . 10
6.2 Transverse-field regularization . . . 11

7 VQE Formulation **11**
7.1 Variational principle . . . 11
7.2 Why a diagonal cost Hamiltonian is still useful . . . 11
7.3 Proposed ansatz families . . . 11
7.3.1 Hardware-efficient ansatz . . . 12
7.3.2 Hamiltonian variational ansatz . . . 12

8 QuLacs Implementation Guide **12**
8.1 Software environment . . . 12
8.2 Recommended repository structure . . . 12
8.3 Generating the synthetic network . . . 13
8.4 Defining scenarios . . . 14
8.5 Building the action-effectiveness tensor . . . 14
8.6 Constructing the QUBO . . . 15
8.7 Mapping QUBO to Ising coefficients . . . 16
8.8 Constructing a QuLacs observable . . . 16
8.9 QAOA-like Hamiltonian variational circuit . . . 17
8.10 Hardware-efficient circuit . . . 18
8.11 Classical optimizer . . . 19
8.12 Sampling and decoding bitstrings . . . 19

9 Classical Post-Processing and Baselines **20**
9.1 Greedy baseline . . . 20
9.2 Single-bit local search . . . 20
9.3 Simulated annealing baseline . . . 21

10 Metrics and Reporting **22**
10.1 Portfolio metrics . . . 22
10.2 Scenario robustness . . . 22
10.3 Quantum metrics . . . 22
10.4 Approximation ratio . . . 23

11 Validation Plan **23**
11.1 Unit tests . . . 23
11.2 QUBO-Ising consistency test . . . 23
11.3 Small-instance exact verification . . . 24

12 Experiment Plan **24**
12.1 Experiment 1: QUBO construction sanity check 24
12.2 Experiment 2: Small exact benchmark 24
12.3 Experiment 3: Full 30-qubit run 24
12.4 Experiment 4: Robustness under shocks 25
12.5 Experiment 5: Fujitsu simulator scaling report 25

13 Interpretation for Corporate Leaders **25**

14 Risk Register **26**

15 Minimum Viable Demonstration **26**

16 Expected Outputs **26**

1 Brief Intro.

This document specifies the technically safest version of the project originally motivated by a quantum-enhanced early-warning idea. The safe version must be understood as a *scenario-optimization simulator*, not as a literal tool for predicting World War III or prescribing real geopolitical interventions.

The central computational task is:

> Given a synthetic network of geopolitical tensions and a set of candidate civilian de-escalation measures, find low-cost action portfolios that reduce a modeled escalation-risk proxy across multiple plausible scenarios.

The quantum part is a VQE-based optimizer for a binary portfolio problem. Each qubit represents a candidate action, not a real country, person, or operational target. The model can use a country-like signed network to generate the QUBO coefficients, but the quantum register is most safely assigned to action-selection variables. This design makes the project technically cleaner, ethically safer, and easier to validate.

The practical deliverable is a benchmarkable hybrid quantum-classical workflow:

Step 1. Generate a synthetic signed country-like network.

Step 2. Define stress scenarios and civilian de-escalation measures.

Step 3. Convert scenario-weighted residual tension into a quadratic risk proxy.

Step 4. Add budget and feasibility penalties.

Step 5. Map the QUBO to an Ising Hamiltonian.

Step 6. Run VQE or QAOA-style VQE using QuLacs.

Step 7. Decode sampled bitstrings into action portfolios.

Step 8. Refine and benchmark with classical optimization.

Step 9. Report risk-proxy reduction, budget use, robustness, and runtime.

This is the version that can actually be implemented and defended.

2 Scope: What This Project Does and Does Not Do

2.1 What the project does

The project develops a quantum-assisted simulator for *abstract de-escalation portfolio optimization*. The workflow is designed to answer questions of the following type:

> Under a synthetic stress scenario, and under a fixed resource budget, which combination of civilian confidence-building actions minimizes a specified model risk proxy?

Examples of safe action categories are:

- crisis hotlines and incident-prevention protocols,
- verified communication channels,

- third-party monitoring or transparency mechanisms,
- structured diplomatic dialogue mechanisms,
- misinformation resilience and public communication support,
- humanitarian coordination channels.

The quantum simulator is used to explore the resulting binary optimization landscape. The model is not expected to outperform all classical methods. Rather, the project asks whether Fujitsu-scale quantum simulation can provide a useful and scalable research environment for structured, frustrated, combinatorial decision models.

2.2 What the project does not do

The project must not claim the following:

- It does not predict World War III.
- It does not guarantee prevention of war.
- It does not produce operational national-security recommendations.
- It does not infer real diplomatic strategy from public data alone.
- It does not treat the Ising ground state as literal world peace.

All claims should be framed as model-internal. For example, a low-energy state means low value of the chosen optimization objective, not real-world peace. A risk score is a model risk proxy, not a validated early-warning index.

3 Proposed Project Architecture

3.1 Primary modeling choice

There are two possible mappings:

(a) **Actor-state mapping:** each qubit represents a country or actor state.

(b) **Action-portfolio mapping:** each qubit represents a candidate preventive action.

Let us try the action-portfolio mapping as the primary deliverable. The actor-state mapping is useful pedagogically, but it is harder to interpret safely. If a qubit represents a country decision, the output may be misread as a geopolitical prediction. If a qubit represents a nonviolent intervention option, the output is a standard portfolio recommendation under a model.

Thus, for the safe version:

$$\text{qubit } k \quad \longleftrightarrow \quad \text{candidate de-escalation action } k.$$

For a 30-qubit implementation:

$$M = 30,$$

where M is the number of candidate actions.

3.2 Data layers

The project has three data layers:

1. **Synthetic country network:** a signed weighted network of abstract regions or blocs.
2. **Scenario layer:** stressors that amplify selected tensions.
3. **Action layer:** civilian actions that reduce specified tensions with a given effectiveness model.

The synthetic network generates the coefficients of the action-portfolio QUBO. The VQE optimizer then searches over binary action portfolios.

4 Mathematical Model

4.1 Synthetic country network

Let the abstract country or geopolitical system contain N_A regions or actors:

$$V = \{1, 2, \ldots, N_A\}.$$

For the recommended toy model:

$$N_A = 30.$$

Let $J_{ij} \in \mathbb{R}$ be a signed interaction weight:

- $J_{ij} > 0$ means cooperative or stabilizing relation,
- $J_{ij} < 0$ means antagonistic or destabilizing relation,
- $|J_{ij}|$ is the strength of the relation.

Define the nonnegative tension weight

$$t_{ij} = \max(0, -J_{ij}).$$

Only negative ties contribute to the baseline tension proxy.

4.2 Block-structured country generator

Partition the 30 actors into $K = 5$ macro-regions, each containing 6 actors:

$$\mathcal{B}_1 = \{1, \ldots, 6\}, \quad \mathcal{B}_2 = \{7, \ldots, 12\}, \quad \ldots, \quad \mathcal{B}_5 = \{25, \ldots, 30\}.$$

A simple signed stochastic block model is:

$$J_{ij} \sim \begin{cases} +\mathrm{Unif}(J_{\mathrm{in,min}}, J_{\mathrm{in,max}}), & b(i) = b(j), \\ -\mathrm{Unif}(J_{\mathrm{out,min}}, J_{\mathrm{out,max}}), & b(i) \neq b(j) \text{ with probability } p_{\mathrm{neg}}, \\ +\mathrm{Unif}(0, J_{\mathrm{out,pos}}), & b(i) \neq b(j) \text{ with probability } 1 - p_{\mathrm{neg}}. \end{cases}$$

Recommended parameters:

$$J_{\mathrm{in,min}} = 0.8, \quad J_{\mathrm{in,max}} = 1.2,$$

$$J_{\text{out,min}} = 0.2, \quad J_{\text{out,max}} = 0.8,$$
$$p_{\text{neg}} = 0.7, \quad J_{\text{out,pos}} = 0.2.$$

To mimic geography, arrange the five macro-regions on a ring and increase the interaction density between neighboring blocks:

$$(1,2), (2,3), (3,4), (4,5), (5,1).$$

This generator creates a network with local cohesion, cross-regional antagonism, and frustration cycles.

4.3 Scenario set

Let $\mathcal{S}$ be a finite set of stress scenarios. For the first implementation, use 10 scenarios:

- 5 border-stress scenarios between neighboring macro-regions,
- 5 internal-stress scenarios, one per macro-region.

Let $w_s \geq 0$ be the planning weight for scenario s, with

$$\sum_{s \in \mathcal{S}} w_s = 1.$$

Recommended values:

$$w_s = 0.12 \quad \text{for each border-stress scenario,}$$
$$w_s = 0.08 \quad \text{for each internal-stress scenario.}$$

Scenario-dependent tensions are defined as follows. For a border scenario involving macro-regions a and b:

$$t_{ij}^{(s)} = \begin{cases} (1 + \delta_{\text{border}}) t_{ij}, & i \in \mathcal{B}_a,\ j \in \mathcal{B}_b, \\ t_{ij}, & \text{otherwise.} \end{cases}$$

For an internal shock in macro-region a:

$$t_{ij}^{(s)} = \begin{cases} (1 + \delta_{\text{internal}}) t_{ij}, & i, j \in \mathcal{B}_a, \\ t_{ij}, & \text{otherwise.} \end{cases}$$

Recommended values:

$$\delta_{\text{border}} = 1.0, \qquad \delta_{\text{internal}} = 0.7.$$

4.4 Candidate de-escalation actions

Let there be $M = 30$ candidate actions:

$$x_k \in \{0, 1\}, \quad k = 1, \ldots, M.$$

Each macro-region has six possible nonviolent actions:

A1. crisis hotline and incident-prevention protocol,

A2. verified communication channel,

A3. third-party observation or verification mechanism,

A4. structured dialogue process,

A5. misinformation resilience and public communication support,

A6. humanitarian coordination channel.

Thus:

$$M = 5 \times 6 = 30.$$

Each action k has a cost $c_k > 0$ and edge-effectiveness coefficients $\alpha_{k,ij} \in [0, 1]$.

4.5 Action effectiveness matrix

If action k belongs to macro-region r, define

$$\alpha_{k,ij} = \begin{cases} a_k^{\text{in}}, & i, j \in \mathcal{B}_r, \\ a_k^{\text{out}}, & i \in \mathcal{B}_r,\ j \notin \mathcal{B}_r,\ b(j) \in \{r-1, r+1\}, \\ a_k^{\text{out}}, & j \in \mathcal{B}_r,\ i \notin \mathcal{B}_r,\ b(i) \in \{r-1, r+1\}, \\ 0, & \text{otherwise.} \end{cases}$$

The block indices are interpreted modulo 5.

Recommended effectiveness values are:

Action type	a^{in}	a^{out}
Hotline and incident protocol	0.25	0.10
Verified communication channel	0.22	0.10
Observation or verification	0.18	0.08
Structured dialogue	0.16	0.07
Misinformation resilience	0.15	0.05
Humanitarian coordination	0.12	0.05

The coefficients should be interpreted as simulation parameters, not empirical causal estimates.

5 From Scenario Risk to QUBO

5.1 Residual tension model

Under scenario s, the residual tension on edge (i, j) after applying action portfolio $\mathbf{x}$ is modeled as

$$t_{ij}^{(s)} \prod_{k=1}^{M} (1 - \alpha_{k,ij} x_k).$$

The scenario risk proxy is

$$r_s(\mathbf{x}) = \sum_{i<j} t_{ij}^{(s)} \prod_{k=1}^{M} (1 - \alpha_{k,ij} x_k).$$

This captures diminishing returns. If two actions affect the same edge, their combined effect is not simply additive.

5.2 Second-order QUBO approximation

For small to moderate $\alpha_{k,ij}$, expand the product to second order:

$$\prod_k (1 - \alpha_{k,ij} x_k) \approx 1 - \sum_k \alpha_{k,ij} x_k + \sum_{k<\ell} \alpha_{k,ij} \alpha_{\ell,ij} x_k x_\ell.$$

Therefore,

$$r_s(\mathbf{x}) \approx A_s + \sum_k b_{s,k} x_k + \sum_{k<\ell} d_{s,k\ell} x_k x_\ell,$$

where

$$A_s = \sum_{i<j} t_{ij}^{(s)},$$

$$b_{s,k} = -\sum_{i<j} t_{ij}^{(s)} \alpha_{k,ij},$$

$$d_{s,k\ell} = \sum_{i<j} t_{ij}^{(s)} \alpha_{k,ij} \alpha_{\ell,ij}.$$

The linear coefficient $b_{s,k}$ is negative when action k reduces risk. The quadratic coefficient $d_{s,k\ell}$ is typically positive because overlapping actions have diminishing returns.

5.3 Scenario aggregation

The expected risk proxy is

$$R(\mathbf{x}) = \sum_{s \in \mathcal{S}} w_s r_s(\mathbf{x}).$$

Thus

$$R(\mathbf{x}) = A + \sum_k B_k x_k + \sum_{k<\ell} D_{k\ell} x_k x_\ell,$$

where

$$A = \sum_s w_s A_s,$$

$$B_k = \sum_s w_s b_{s,k},$$

$$D_{k\ell} = \sum_s w_s d_{s,k\ell}.$$

5.4 Budget penalty

Let c_k be the cost of action k and B_{budget} be the target budget. Use a soft penalty

$$P(\mathbf{x}) = \eta \left(\sum_k c_k x_k - B_{\text{budget}} \right)^2.$$

Since $x_k^2 = x_k$, expand:

$$P(\mathbf{x}) = \eta \left[\sum_k c_k^2 x_k + 2 \sum_{k<\ell} c_k c_\ell x_k x_\ell - 2 B_{\text{budget}} \sum_k c_k x_k + B_{\text{budget}}^2 \right].$$

The final QUBO objective is

$$F(\mathbf{x}) = R(\mathbf{x}) + P(\mathbf{x}).$$

Ignoring the constant, write

$$F(\mathbf{x}) = \sum_k q_k x_k + \sum_{k<\ell} q_{k\ell} x_k x_\ell + \text{const},$$

with

$$q_k = B_k + \eta(c_k^2 - 2B_{\text{budget}} c_k),$$
$$q_{k\ell} = D_{k\ell} + 2\eta c_k c_\ell.$$

5.5 Optional incompatibility penalties

If two actions cannot be simultaneously selected, add

$$\Lambda x_k x_\ell$$

for that pair. This modifies

$$q_{k\ell} \leftarrow q_{k\ell} + \Lambda.$$

The penalty Λ should be chosen larger than the typical risk-reduction benefit of selecting both actions.

6 QUBO to Ising Hamiltonian

6.1 Binary-spin mapping

Use

$$x_k = \frac{1 - Z_k}{2},$$

where Z_k is the Pauli-Z operator on qubit k.

For a QUBO

$$F(\mathbf{x}) = \sum_k q_k x_k + \sum_{k<\ell} q_{k\ell} x_k x_\ell + \text{const},$$

we have

$$x_k x_\ell = \frac{1 - Z_k - Z_\ell + Z_k Z_\ell}{4}.$$

Therefore the Ising cost Hamiltonian is

$$\hat{H}_C = \sum_{k<\ell} K_{k\ell} Z_k Z_\ell + \sum_k g_k Z_k + cI,$$

with

$$K_{k\ell} = \frac{q_{k\ell}}{4},$$
$$g_k = -\frac{q_k}{2} - \sum_{\ell \neq k} \frac{q_{k\ell}}{4},$$

where $q_{k\ell}$ is understood symmetrically, and

$$c = \text{const} + \frac{1}{2}\sum_k q_k + \frac{1}{4}\sum_{k<\ell} q_{k\ell}.$$

The constant c does not affect the minimizing bitstring, but it is useful for reporting absolute objective values.

6.2 Transverse-field regularization

For a VQE simulation, define

$$\hat{H}(\Gamma) = \hat{H}_C - \Gamma \sum_{k=1}^{M} X_k.$$

The transverse field serves two roles:

1. It turns the purely classical diagonal problem into a noncommuting quantum Hamiltonian.
2. It encourages superposition over nearby action portfolios during optimization.

The final output portfolios should still be evaluated using the original classical objective $F(\mathbf{x})$. The transverse term is an exploration device, not part of the final policy score unless explicitly stated.

7 VQE Formulation

7.1 Variational principle

Given a parameterized quantum state

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle^{\otimes M},$$

VQE minimizes

$$E(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta})|\hat{H}|\psi(\boldsymbol{\theta})\rangle.$$

The optimizer returns parameters

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} E(\boldsymbol{\theta}).$$

Sampling the optimized state gives bitstrings

$$b_1 b_2 \dots b_M,$$

which are decoded as action decisions

$$x_k = b_k.$$

7.2 Why a diagonal cost Hamiltonian is still useful

Although $\hat{H}_C$ is diagonal, the variational circuit can produce nontrivial probability distributions over portfolios. The diagonal Hamiltonian also gives efficient classical evaluation of sampled bitstrings. However, because diagonal QUBO problems often admit strong classical baselines, the project must benchmark fairly against classical algorithms.

7.3 Proposed ansatz families

Let's use two ansatz families.

7.3.1 Hardware-efficient ansatz

A simple hardware-efficient ansatz is

$$U_{\mathrm{HEA}}(\boldsymbol{\theta}) = \prod_{\ell=1}^{p} \left[U_{\mathrm{ent}} \prod_{k=1}^{M} R_z(\theta_{\ell,k}^{(z)}) R_y(\theta_{\ell,k}^{(y)}) \right] \prod_{k=1}^{M} R_y(\theta_{0,k}).$$

The entangling block may be a ring of CNOT gates:

$$(0 \to 1), (1 \to 2), \ldots, (M-2 \to M-1), (M-1 \to 0).$$

This ansatz is easy to implement and serves as a baseline.

7.3.2 Hamiltonian variational ansatz

A more structure-aware ansatz is QAOA-like:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{r=1}^{p} e^{-i\beta_r H_M} e^{-i\gamma_r H_C} |+\rangle^{\otimes M},$$

where

$$H_M = \sum_{k=1}^{M} X_k.$$

This can be interpreted as a VQE ansatz for $\hat{H}(\Gamma)$ because the variational state is optimized with respect to the full target Hamiltonian.

8 QuLacs Implementation Guide

8.1 Software environment

The minimal local environment seems to be:

```
python>=3.10
numpy
scipy
pandas
matplotlib
qulacs
```

Fujitsu QARP APIs may provide higher-level abstractions. This note uses QuLacs-level code because it is transparent and portable. When QARP-specific VQE modules are available, the same Hamiltonian coefficients and circuit definitions can be passed to those modules.

8.2 Recommended repository structure

```
qe_ews_safe_vqe/
  README.md
  requirements.txt
  configs/
    default.yaml
  src/
```

```
    generate_network.py
    scenarios.py
    actions.py
    qubo.py
    ising.py
    qulacs_vqe.py
    baselines.py
    metrics.py
    plotting.py
  notebooks/
    01_generate_instance.ipynb
    02_verify_qubo.ipynb
    03_run_vqe.ipynb
    04_compare_baselines.ipynb
  results/
    raw/
    figures/
    tables/
```

8.3 Generating the synthetic network

```python
import numpy as np


def generate_country_network(
    n_blocks=5,
    block_size=6,
    p_in=0.8,
    p_out=0.25,
    p_neg=0.7,
    j_in=(0.8, 1.2),
    j_out=(0.2, 0.8),
    j_out_pos=0.2,
    seed=123,
):
    rng = np.random.default_rng(seed)
    n = n_blocks * block_size
    block = np.repeat(np.arange(n_blocks), block_size)
    J = np.zeros((n, n), dtype=float)

    for i in range(n):
        for j in range(i + 1, n):
            same = block[i] == block[j]
            if same:
                if rng.random() < p_in:
                    val = rng.uniform(*j_in)
                else:
                    val = 0.0
            else:
                if rng.random() < p_out:
                    if rng.random() < p_neg:
                        val = -rng.uniform(*j_out)
                    else:
```

```
                        val = rng.uniform(0.0, j_out_pos)
                else:
                    val = 0.0
            J[i, j] = J[j, i] = val
    return J, block


def tension_matrix(J):
    return np.maximum(0.0, -J)
```

8.4 Defining scenarios

```
def build_scenarios(T, block, delta_border=1.0, delta_internal=0.7):
    n_blocks = int(block.max()) + 1
    scenarios = []

    # Border scenarios on a ring.
    for a in range(n_blocks):
        b = (a + 1) % n_blocks
        Ts = T.copy()
        mask_a = np.where(block == a)[0]
        mask_b = np.where(block == b)[0]
        for i in mask_a:
            for j in mask_b:
                Ts[i, j] *= (1.0 + delta_border)
                Ts[j, i] *= (1.0 + delta_border)
        scenarios.append({"name": f"border_{a}_{b}", "weight": 0.12, "T": Ts})

    # Internal shock scenarios.
    for a in range(n_blocks):
        Ts = T.copy()
        mask = np.where(block == a)[0]
        for idx_i, i in enumerate(mask):
            for j in mask[idx_i + 1:]:
                Ts[i, j] *= (1.0 + delta_internal)
                Ts[j, i] *= (1.0 + delta_internal)
        scenarios.append({"name": f"internal_{a}", "weight": 0.08, "T": Ts})

    total_w = sum(s["weight"] for s in scenarios)
    for s in scenarios:
        s["weight"] /= total_w
    return scenarios
```

8.5 Building the action-effectiveness tensor

```
def build_action_effectiveness(block):
    n = len(block)
    n_blocks = int(block.max()) + 1
    actions_per_block = 6
    m = n_blocks * actions_per_block
```

```
    # (a_in, a_out) for six action types.
    eff = [
        (0.25, 0.10),
        (0.22, 0.10),
        (0.18, 0.08),
        (0.16, 0.07),
        (0.15, 0.05),
        (0.12, 0.05),
    ]

    alpha = np.zeros((m, n, n), dtype=float)

    for r in range(n_blocks):
        left = (r - 1) % n_blocks
        right = (r + 1) % n_blocks
        for a_type, (a_in, a_out) in enumerate(eff):
            k = r * actions_per_block + a_type
            for i in range(n):
                for j in range(i + 1, n):
                    bi, bj = block[i], block[j]
                    val = 0.0
                    if bi == r and bj == r:
                        val = a_in
                    elif (bi == r and bj in [left, right]) or (bj == r and bi in [left,
    right]):
                        val = a_out
                    alpha[k, i, j] = alpha[k, j, i] = val
    return alpha
```

8.6 Constructing the QUBO

```
def build_qubo(scenarios, alpha, costs, budget, eta):
    m = alpha.shape[0]
    B_lin = np.zeros(m)
    D_quad = np.zeros((m, m))
    A_const = 0.0

    for sc in scenarios:
        w = sc["weight"]
        T = sc["T"]
        n = T.shape[0]

        A_s = 0.0
        b_s = np.zeros(m)
        d_s = np.zeros((m, m))

        for i in range(n):
            for j in range(i + 1, n):
                tij = T[i, j]
                if tij == 0.0:
                    continue
                A_s += tij
                a_vec = alpha[:, i, j]
```

```
                b_s -= tij * a_vec
                for k in range(m):
                    if a_vec[k] == 0.0:
                        continue
                    for ell in range(k + 1, m):
                        if a_vec[ell] != 0.0:
                            d_s[k, ell] += tij * a_vec[k] * a_vec[ell]

        A_const += w * A_s
        B_lin += w * b_s
        D_quad += w * d_s

    q = B_lin + eta * (costs**2 - 2.0 * budget * costs)
    Q = D_quad.copy()
    for k in range(m):
        for ell in range(k + 1, m):
            Q[k, ell] += 2.0 * eta * costs[k] * costs[ell]

    const = A_const + eta * budget**2
    return q, Q, const
```

8.7 Mapping QUBO to Ising coefficients

```
def qubo_to_ising(q, Q, const=0.0):
    m = len(q)
    K = np.zeros((m, m), dtype=float)
    g = np.zeros(m, dtype=float)
    c = const + 0.5 * np.sum(q)

    for k in range(m):
        g[k] += -0.5 * q[k]

    for k in range(m):
        for ell in range(k + 1, m):
            qkl = Q[k, ell]
            K[k, ell] = K[ell, k] = 0.25 * qkl
            g[k] += -0.25 * qkl
            g[ell] += -0.25 * qkl
            c += 0.25 * qkl
    return K, g, c
```

8.8 Constructing a QuLacs observable

QuLacs represents observables as sums of Pauli strings. The cost Hamiltonian plus transverse field is

$$\hat{H} = \sum_{k<\ell} K_{k\ell} Z_k Z_\ell + \sum_k g_k Z_k - \Gamma \sum_k X_k.$$

```
from qulacs import Observable


def build_observable(K, g, gamma_tf=0.0):
```

```
    m = len(g)
    obs = Observable(m)

    for k in range(m):
        if abs(g[k]) > 1e-14:
            obs.add_operator(float(g[k]), f"Z {k}")

    for k in range(m):
        for ell in range(k + 1, m):
            if abs(K[k, ell]) > 1e-14:
                obs.add_operator(float(K[k, ell]), f"Z {k} Z {ell}")

    if abs(gamma_tf) > 1e-14:
        for k in range(m):
            obs.add_operator(float(-gamma_tf), f"X {k}")

    return obs
```

8.9 QAOA-like Hamiltonian variational circuit

The circuit implements

$$\prod_{r=1}^{p} e^{-i\beta_r \sum_k X_k} e^{-i\gamma_r H_C} |+\rangle^{\otimes M}.$$

For a Z_k term,

$$e^{-i\gamma g_k Z_k} = R_z(2\gamma g_k).$$

For a $Z_k Z_\ell$ term,

$$e^{-i\gamma K_{k\ell} Z_k Z_\ell}$$

can be implemented using CNOT, R_z, CNOT:

$$\mathrm{CNOT}_{k\to\ell} R_z(2\gamma K_{k\ell})_\ell \mathrm{CNOT}_{k\to\ell}.$$

For the mixer,

$$e^{-i\beta X_k} = R_x(2\beta).$$

```
from qulacs import QuantumCircuit, QuantumState


def build_hva_circuit(K, g, params):
    """
    params = [gamma_0, ..., gamma_{p-1}, beta_0, ..., beta_{p-1}]
    """
    m = len(g)
    p = len(params) // 2
    gammas = params[:p]
    betas = params[p:]

    circuit = QuantumCircuit(m)

    # Initial |+> state.
    for k in range(m):
```

```
        circuit.add_H_gate(k)

    for r in range(p):
        gamma = gammas[r]
        beta = betas[r]

        # Cost phase: linear Z terms.
        for k in range(m):
            angle = 2.0 * gamma * g[k]
            if abs(angle) > 1e-14:
                circuit.add_RZ_gate(k, angle)

        # Cost phase: ZZ terms.
        for k in range(m):
            for ell in range(k + 1, m):
                angle = 2.0 * gamma * K[k, ell]
                if abs(angle) > 1e-14:
                    circuit.add_CNOT_gate(k, ell)
                    circuit.add_RZ_gate(ell, angle)
                    circuit.add_CNOT_gate(k, ell)

        # Mixer.
        for k in range(m):
            circuit.add_RX_gate(k, 2.0 * beta)

    return circuit


def evaluate_energy_hva(params, K, g, observable):
    m = len(g)
    state = QuantumState(m)
    state.set_zero_state()
    circuit = build_hva_circuit(K, g, params)
    circuit.update_quantum_state(state)
    return observable.get_expectation_value(state).real
```

8.10 Hardware-efficient circuit

```
def build_hea_circuit(m, params, depth):
    """
    params length = m + 2*m*depth.
    First m parameters initialize RY rotations.
    Each layer uses RY and RZ on every qubit plus ring CNOTs.
    """
    circuit = QuantumCircuit(m)
    idx = 0

    for k in range(m):
        circuit.add_RY_gate(k, params[idx])
        idx += 1

    for layer in range(depth):
        for k in range(m):
```

```
            circuit.add_RY_gate(k, params[idx])
            idx += 1
            circuit.add_RZ_gate(k, params[idx])
            idx += 1

        # Ring entanglement.
        for k in range(m - 1):
            circuit.add_CNOT_gate(k, k + 1)
        circuit.add_CNOT_gate(m - 1, 0)

    return circuit


def evaluate_energy_hea(params, m, depth, observable):
    state = QuantumState(m)
    state.set_zero_state()
    circuit = build_hea_circuit(m, params, depth)
    circuit.update_quantum_state(state)
    return observable.get_expectation_value(state).real
```

8.11 Classical optimizer

For the first implementation, use derivative-free methods. COBYLA is robust for small parameter counts. Nelder-Mead is also useful. For HVA with depth p, the number of parameters is only $2p$.

```
from scipy.optimize import minimize
import numpy as np


def run_hva_vqe(K, g, gamma_tf=0.2, depth=2, seed=123):
    rng = np.random.default_rng(seed)
    obs = build_observable(K, g, gamma_tf=gamma_tf)

    x0 = rng.uniform(low=-0.1, high=0.1, size=2 * depth)

    def objective(params):
        return evaluate_energy_hva(params, K, g, obs)

    res = minimize(
        objective,
        x0,
        method="COBYLA",
        options={"maxiter": 300, "rhobeg": 0.5, "tol": 1e-4},
    )
    return res
```

8.12 Sampling and decoding bitstrings

After optimization, sample bitstrings from the final state.

```
def sample_hva_portfolios(K, g, params, n_shots=2000):
    m = len(g)
    state = QuantumState(m)
```

```
    state.set_zero_state()
    circuit = build_hva_circuit(K, g, params)
    circuit.update_quantum_state(state)

    samples = state.sampling(n_shots)
    X = np.zeros((n_shots, m), dtype=int)
    for r, integer_sample in enumerate(samples):
        for k in range(m):
            X[r, k] = (integer_sample >> k) & 1
    return X


def qubo_value(x, q, Q, const=0.0):
    val = const + float(np.dot(q, x))
    m = len(x)
    for k in range(m):
        for ell in range(k + 1, m):
            val += Q[k, ell] * x[k] * x[ell]
    return val


def best_sampled_portfolios(samples, q, Q, const=0.0, top_k=10):
    unique = {}
    for x in samples:
        key = tuple(int(v) for v in x)
        if key not in unique:
            unique[key] = qubo_value(x, q, Q, const)
    ranked = sorted(unique.items(), key=lambda kv: kv[1])
    return ranked[:top_k]
```

9 Classical Post-Processing and Baselines

9.1 Greedy baseline

A simple greedy baseline ranks actions by benefit per cost. Approximate marginal benefit is $-B_k/c_k$ or $-q_k/c_k$, with care because budget penalties alter q_k.

9.2 Single-bit local search

For QUBO

$$F(\mathbf{x}) = \sum_k q_k x_k + \sum_{k<\ell} q_{k\ell} x_k x_\ell,$$

flipping bit k changes the objective by

$$\Delta_k = (1 - 2x_k) \left(q_k + \sum_{\ell \neq k} q_{k\ell} x_\ell \right).$$

If $\Delta_k < 0$, the flip improves the objective.

```
def local_search(x, q, Q, max_sweeps=100):
    x = x.copy().astype(int)
```

```
    m = len(x)
    for _ in range(max_sweeps):
        improved = False
        for k in range(m):
            field = q[k]
            for ell in range(m):
                if ell == k:
                    continue
                a, b = min(k, ell), max(k, ell)
                field += Q[a, b] * x[ell]
            delta = (1 - 2 * x[k]) * field
            if delta < -1e-12:
                x[k] = 1 - x[k]
                improved = True
        if not improved:
            break
    return x
```

9.3 Simulated annealing baseline

```
def simulated_annealing(q, Q, const=0.0, n_steps=20000, t0=1.0, tf=1e-3, seed=123):
    rng = np.random.default_rng(seed)
    m = len(q)
    x = rng.integers(0, 2, size=m)
    fx = qubo_value(x, q, Q, const)
    best_x = x.copy()
    best_fx = fx

    for step in range(n_steps):
        frac = step / max(1, n_steps - 1)
        temp = t0 * (tf / t0) ** frac
        k = rng.integers(0, m)
        x_new = x.copy()
        x_new[k] = 1 - x_new[k]
        f_new = qubo_value(x_new, q, Q, const)
        delta = f_new - fx
        if delta < 0 or rng.random() < np.exp(-delta / max(temp, 1e-12)):
            x, fx = x_new, f_new
            if fx < best_fx:
                best_x, best_fx = x.copy(), fx
    return best_x, best_fx
```

10 Metrics and Reporting

10.1 Portfolio metrics

For a portfolio $\mathbf{x}$, report:

$$\text{Budget}(\mathbf{x}) = \sum_k c_k x_k, \tag{1}$$

$$\text{Risk}(\mathbf{x}) = R(\mathbf{x}), \tag{2}$$

$$\text{Penalty}(\mathbf{x}) = P(\mathbf{x}), \tag{3}$$

$$F(\mathbf{x}) = R(\mathbf{x}) + P(\mathbf{x}). \tag{4}$$

Report also the number of selected actions:

$$N_{\text{selected}} = \sum_k x_k.$$

10.2 Scenario robustness

Compute per-scenario risks:

$$r_s(\mathbf{x}).$$

Then report:

$$\text{MeanRisk}(\mathbf{x}) = \sum_s w_s r_s(\mathbf{x}), \tag{5}$$

$$\text{WorstRisk}(\mathbf{x}) = \max_s r_s(\mathbf{x}), \tag{6}$$

$$\text{RiskStd}(\mathbf{x}) = \sqrt{\sum_s w_s \left(r_s(\mathbf{x}) - R(\mathbf{x})\right)^2}. \tag{7}$$

For corporate reporting, use plots:

- risk reduction versus budget,
- VQE energy convergence,
- sampled portfolio objective histogram,
- robustness boxplot across scenarios,
- runtime versus qubit count and ansatz depth.

10.3 Quantum metrics

Report:

- number of qubits M,
- ansatz type,
- depth p,
- number of variational parameters,

- number of Hamiltonian terms,
- number of optimizer iterations,
- final expectation value,
- best sampled classical objective,
- approximation ratio if a classical optimum is known for small instances.

10.4 Approximation ratio

For small instances where exact optimum F^* is available, define

$$\rho = \frac{F(\mathbf{x}_{\text{alg}}) - F_{\text{worst}}}{F^* - F_{\text{worst}}}.$$

Because the sign convention may vary, define the ratio carefully in the final report. A simpler alternative is to report the relative optimality gap:

$$\text{gap} = \frac{F(\mathbf{x}_{\text{alg}}) - F^*}{|F^*| + 10^{-12}}.$$

11 Validation Plan

11.1 Unit tests

Before running VQE, verify:

1. QUBO values match Ising values for random bitstrings.
2. Local search decreases the QUBO objective.
3. The QuLacs observable expectation on computational basis states matches the Ising objective.
4. The budget penalty behaves correctly.

11.2 QUBO-Ising consistency test

For a bitstring $\mathbf{x}$, define spins

$$z_k = 1 - 2x_k.$$

Then verify

$$F(\mathbf{x}) = \sum_{k<\ell} K_{k\ell} z_k z_\ell + \sum_k g_k z_k + c.$$

```
def ising_value_from_bits(x, K, g, c=0.0):
    z = 1 - 2 * np.asarray(x)
    val = c + np.dot(g, z)
    m = len(z)
    for k in range(m):
        for ell in range(k + 1, m):
            val += K[k, ell] * z[k] * z[ell]
    return float(val)
```

11.3 Small-instance exact verification

For $M \leq 20$, brute force is feasible:

$$2^{20} = 1,048,576.$$

Use this to validate the full pipeline before running $M = 30$.

For $M = 30$, brute force is not recommended:

$$2^{30} \approx 1.07 \times 10^9.$$

Use simulated annealing, greedy baselines, and local search instead.

12 Experiment Plan

12.1 Experiment 1: QUBO construction sanity check

Goal: verify that the synthetic network produces nontrivial tension and action effects.

Outputs:

- histogram of J_{ij},
- heatmap of t_{ij},
- top actions by marginal risk reduction,
- budget penalty curve.

12.2 Experiment 2: Small exact benchmark

Use $M = 12$, 18, and 20 actions. Compare:

- exact optimum,
- greedy,
- simulated annealing,
- VQE with HEA,
- VQE with HVA.

12.3 Experiment 3: Full 30-qubit run

Use $M = 30$ action qubits. Run:

$$p \in \{1, 2, 3\}, \quad \Gamma \in \{0, 0.1, 0.2, 0.5\}.$$

For each setting, record:

- best sampled portfolio,
- mean of top 10 sampled portfolios,
- budget use,
- risk reduction,
- runtime,
- optimizer convergence.

12.4 Experiment 4: Robustness under shocks

Perturb the scenario tensions:

$$t_{ij}^{(s)} \mapsto t_{ij}^{(s)}(1 + \epsilon_{ij}),$$

where

$$\epsilon_{ij} \sim \text{Normal}(0, \sigma^2).$$

Evaluate whether the same portfolios remain effective.

12.5 Experiment 5: Fujitsu simulator scaling report

Measure performance as a function of:

- qubit count,
- number of Pauli terms,
- ansatz depth,
- number of optimizer iterations,
- circuit entanglement pattern.

This directly supports the challenge objective of testing simulator capability on a novel problem.

13 Interpretation for Corporate Leaders

After we run all the calculations and obtain the intended/(or any unintended) results, we might need to communicate the project as follows:

> We use Fujitsu-scale quantum simulation to benchmark whether variational quantum methods can support complex scenario optimization for civilian de-escalation planning. The output is not a geopolitical prediction. It is a structured comparison of possible action portfolios under explicitly stated assumptions.

The key value propositions are:

1. **Novel domain:** applies quantum simulation beyond chemistry and finance.
2. **Clear optimization structure:** QUBO and Ising mappings are mathematically explicit.
3. **Responsible framing:** avoids operational claims and focuses on scenario planning.
4. **Benchmark value:** produces measurable simulator workloads and classical comparisons.
5. **Business relevance:** similar methods can transfer to supply-chain risk, stakeholder management, crisis communication, and portfolio planning.

14 Risk Register

Risk	Problem	Mitigation
Overclaiming	Reviewers may reject predictive-war claims	Use scenario optimization language only
Classical baselines outperform VQE	Likely for some QUBOs	Present as simulator benchmark and hybrid workflow
Dense Hamiltonian too costly	Many ZZ terms increase circuit depth	Sparsify QUBO and test low-depth HVA
Unclear policy interpretation	Outputs may be misunderstood	Use synthetic actions and non-operational wording
Noisy public data	GDELT-like data may be ambiguous	Use synthetic primary model, public data optional
Budget penalty dominates	QUBO may select trivial portfolios	Tune η and report sensitivity

15 Minimum Viable Demonstration

A credible minimum demonstration consists of:

1. $M = 30$ action variables and 30 qubits.
2. Synthetic signed country network with 30 actors.
3. Ten stress scenarios.
4. Quadratic risk model and budget penalty.
5. QUBO-Ising verification tests.
6. QuLacs VQE with HVA at depths $p = 1, 2, 3$.
7. Greedy and simulated annealing baselines.
8. Final report with risk-reduction curves and best action portfolios.

16 Expected Outputs

The final project should deliver:

- Source code repository.
- Technical report.
- Hamiltonian term files in CSV or JSON.
- Convergence plots.
- Pareto frontier plots.
- Baseline comparison tables.
- A short business-facing slide summary.

References

[1] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, 4213, 2014.

[2] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, 79, 2018.

[3] E. Farhi, J. Goldstone, and S. Gutmann, "A Quantum Approximate Optimization Algorithm," arXiv:1411.4028, 2014.

[4] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, 5, 2014.

[5] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.

[6] Qulacs Documentation, "Qulacs Python API Reference and Advanced Guide," https://docs.qulacs.org/en/latest/, accessed May 26, 2026.

[7] Qulacs GitHub Repository, https://github.com/qulacs/qulacs, accessed May 26, 2026.

[8] Fujitsu, "Fujitsu Quantum Simulator Challenge 2025-26," https://global.fujitsu/en-global/technology/research/article/topics/202512-quantum-simulator-challenge, accessed May 26, 2026.

[9] M. Morita, Y. Tomita, J. Koyama, and K. Kimura, "Simulator Demonstration of Large Scale Variational Quantum Algorithm on HPC Cluster," arXiv:2402.11878, 2024.